

OUTMATE.AI RB2B-STYLE VISITOR IDENTIFICATION FEATURE - COMPLETE AGENT IMPLEMENTATION

🌀 PROJECT OVERVIEW

Build a **production-grade website visitor identification system** (RB2B clone) for Outmate.ai GTM platform. This feature tracks anonymous B2B website visitors via JS pixel, identifies companies/persons using IP enrichment (IPinfo + Enrich.so + Explorium), and pushes real-time alerts to Slack/CRM.

CRITICAL: Integrate with existing ASN Workflows - visitor signals become workflow triggers.

Current Stack (DO NOT CHANGE):

- ☒ **Backend**: FastAPI + Celery + Redis + Postgres (Supabase)
- ☒ **Frontend**: Next.js 15 + shadcn/ui + TanStack Query
- ☒ **Existing Features**: Database Enrichment (Explorium+Crustdata), NLP
- ☐ **Status**: Local-only (test via ngrok)

Budget: \$29/mo (Enrich.so starter) + free tiers

🔒 FINAL TECH STACK (LOCKED - NO CHOICES)

Layer	Tool	Config
Backend	FastAPI 0.112+	`/api/track`, `/api/visitors`
Queue	Celery + Redis	`tasks.process_visitor`
DB	Postgres (Supabase)	`site_configs`, `visits`, `alerts`
Enrichment	IPinfo (free) + Enrich.so (\$29) + Explorium	IP → company → person
Pixel	Custom JS beacon	Vercel CDN (later)
Frontend	Next.js	`/dashboard/visitors` TanStack Table + SSE
Testing	ngrok + Postman	Local pixel testing

📄 DATABASE SCHEMAS (CREATE IMMEDIATELY)

```
``sql
-- 1. Org pixel configurations
CREATE TABLE site_configs (
  org_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  pixel_key VARCHAR(255) UNIQUE NOT NULL,
  domain VARCHAR(255),
  icp_filters JSONB DEFAULT '{}',
  webhook_urls JSONB DEFAULT '[]',
  created_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_site_configs_pixel_key ON site_configs(pixel_key);

-- 2. Visitor tracking + enrichment
CREATE TABLE visits (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id UUID REFERENCES site_configs(org_id) ON DELETE CASCADE,
```

```

ip INET NOT NULL,
url TEXT NOT NULL,
referrer TEXT,
user_agent TEXT,
intent_score FLOAT DEFAULT 0.5,
resolution JSONB,
matched BOOLEAN DEFAULT FALSE,
created_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_visits_org_time ON visits(org_id, created_at DESC);
CREATE INDEX idx_visits_ip_org ON visits(org_id, ip);

-- 3. Webhook audit trail
CREATE TABLE alerts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    visit_id UUID REFERENCES visits(id),
    webhook_type VARCHAR(50),
    status VARCHAR(20),
    payload JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

```



BACKEND IMPLEMENTATION (PHASE 1-2)

1. Dependencies & Config (app/requirements.txt)

```

text
ipinfo-python==5.0.1
requests==2.32.3

```

2. Environment Variables (.env)

```

text
IPINFO_TOKEN=your_free_ipinfo_token
ENRICH_API_KEY=your_enrich_so_key
EXPLORIUM_API_KEY=existing_key
REDIS_URL=your_redis_url

```

3. Core Services (app/services/visitor_enrich.py)

```

python
import ipinfo
from app.services.explorium import explorium_client # Existing

class VisitorEnricher:
    def __init__(self):
        self.ipinfo = ipinfo.Client(settings.IPINFO_TOKEN)

    async def enrich_ip(self, ip: str, url: str, intent_score: float):
        # Free IPinfo company lookup

```

```

company_data = self.ipinfo.org(ip)
resolution = {
    "company": company_data.org,
    "domain": company_data.hostname,
    "geo": company_data.country,
    "confidence": 0.6
}

# High-intent: Enrich.so person lookup
if intent_score > 0.7 or "/pricing" in url or "/demo" in url:
    enrich_response = await self._enrich_so_lookup(ip)
    if enrich_response.get("data", {}).get("email"):
        # Explorium verification (existing service)
        explorium_data = await explorium_client.enrich({
            "email": enrich_response["data"]["email"]
        })
        resolution.update({
            "person": enrich_response["data"],
            "explorium": explorium_data,
            "confidence": 0.85,
            "intent_score": intent_score
        })

    return resolution

async def _enrich_so_lookup(self, ip: str):
    # Enrich.so API call
    response = await httpx.AsyncClient().post(
        "https://api.enrich.so/ip-to-company",
        json={"ip": ip},
        headers={"Authorization": f"Bearer {settings.ENRICH_API_KEY}"})
    return response.json()

```

4. API Endpoints (app/routers/visitors.py)

```

python
@router.post("/track")
async def track_visitor(
    url: str = Form(...),
    referrer: str = Form(None),
    user_agent: str = Header(...),
    x_forwarded_for: str = Header(None),
    pixel_key: str = Header(..., alias="X-Pixel-Key")
):
    org_id = crud.get_org_by_pixel_key(pixel_key)
    if not org_id:
        raise HTTPException(401, "Invalid pixel key")

    ip = x_forwarded_for.split(",").strip() if x_forwarded_for else "127.0.0.1"
    intent_score = 1.0 if any(x in url.lower() for x in ["/pricing", "/demo",
"/contact"]) else 0.5

    # Redis deduplication (1h per IP/domain)
    dedupe_key = f"visits:{org_id}:{ip}:{urlparse(url).netloc}"
    if redis_client.get(dedupe_key):
        return {"status": "deduplicated"}
    redis_client.setex(dedupe_key, 3600, "1")

    # Queue processing
    task = process_visitor.delay(org_id, {
        "ip": ip, "url": url, "referrer": referrer,

```

```

        "user_agent": user_agent, "intent_score": intent_score
    })
    return {"status": "queued", "task_id": task.id}

@router.get("/visitors")
async def list_visitors(org_id: str, limit: int = 100):
    return crud.get_recent_visits(org_id, limit)

```

5. Celery Task (app/tasks/visitors.py)

```

python
@celery.task(bind=True, max_retries=3)
def process_visitor(self, org_id: str, data: dict):
    try:
        enricher = VisitorEnricher()
        resolution = enricher.enrich_ip(data["ip"], data["url"],
data["intent_score"])

        visit = crud.create_visit(org_id, {"**data", "resolution": resolution})
        matched = resolution["confidence"] > 0.7

        if matched:
            crud.update_visit(visit.id, {"matched": True})
            await send_webhooks(org_id, resolution, visit.id)

    except Exception as exc:
        logger.error(f"Visitor processing failed: {exc}")
        raise self.retry(countdown=60)

```

☺ FRONTEND IMPLEMENTATION (PHASE 3)

1. Pixel JS (public/pixel.js)

```

javascript
(function(w,d,k){
    w._outmateVisitor={key:k,track:function(){
        const xhr=new XMLHttpRequest();
        xhr.open('POST','/api/track',true);
        xhr.setRequestHeader('Content-Type','application/json');
        xhr.setRequestHeader('X-Pixel-Key',k);
        xhr.send(JSON.stringify({
            url:location.href,
            referrer:document.referrer,
            userAgent:navigator.userAgent
        }));
    }};
    const s=d.createElement('script');
    s.src=`/pixel.js?key=${k}`;
    d.head.appendChild(s);
    w.addEventListener('load',w._outmateVisitor.track);
    w.addEventListener('pagehide',w._outmateVisitor.track);
})(window,document,'YOUR_PIXEL_KEY');
```

2. Dashboard Page (app/dashboard/visitors/page.tsx)

```

tsx
"use client";
import { useQuery } from '@tanstack/react-query';

```

```

import { DataTable } from '@components/ui/data-table';
import { Badge } from '@components/ui/badge';

const columns = [
  { accessorKey: 'created_at', header: 'Time' },
  { accessorKey: 'company', header: 'Company' },
  {
    accessorKey: 'resolution.person.email',
    header: 'Contact',
    cell: ({ row }) => row.original.resolution?.person?.email || '-'
  },
  {
    accessorKey: 'confidence',
    header: 'Match %',
    cell: ({ row }) => (
      <Badge variant={row.original.matched ? 'default' : 'secondary'}>
        {Math.round(row.original.resolution.confidence * 100)}%
      </Badge>
    )
  }
];

export default function VisitorsPage() {
  const { data: visitors } = useQuery({
    queryKey: ['visitors'],
    queryFn: () => api.get('/api/visitors?limit=100').then(res => res.data)
  });

  return (
    <div className="p-6 space-y-4">
      <div className="grid grid-cols-4 gap-4">
        <Card><CardContent>Today: 42</CardContent></Card>
        <Card><CardContent>Match Rate: 72%</CardContent></Card>
      </div>
      <DataTable columns={columns} data={visitors || []} />
    </div>
  );
}

```

TESTING PROTOCOL (LOCAL COMPLETE)

1. Local Setup

```

bash
# 1. Add schemas to Supabase
# 2. ngrok http 8000
# 3. Create test org: INSERT site_configs (pixel_key) VALUES ('testkey');
# 4. Test pixel: Create HTML with <script src="https://abc.ngrok.io/pixel.js?
key=testkey">

```

2. Test Commands

```

bash
# Test track endpoint
curl -X POST "https://abc.ngrok.io/api/track" \
  -H "X-Pixel-Key: testkey" \
  -H "User-Agent: Mozilla" \
  -d '{"url":"https://example.com/pricing"}'

```

```
# Verify DB
SELECT * FROM visits ORDER BY created_at DESC LIMIT 5;
```

ASN WORKFLOW INTEGRATION

Add to existing workflows:

```
json
{
  "type": "trigger",
  "name": "visitor_identified",
  "config": {
    "confidence_threshold": 0.7,
    "intent_pages": ["/pricing", "/demo"]
  }
}
```

SUCCESS METRICS

- **70%+ match rate** on corporate IPs
- **<1s latency** per track request
- **100% webhook delivery** (retry logic)
- **\$0.005/visit cost**

DELIVERABLES CHECKLIST

- Schemas deployed
- `/api/track` + Celery task working
- `pixel.js` generating visits
- `/dashboard/visitors` page live
- ngrok pixel test confirmed
- Slack webhook test success
- ASN workflow trigger ready